

FamilyWallet

Mobile App — User Stories & Screen Navigation Guide

For AI-Assisted Frontend Build

Version 1.0 | March 2026

Purpose of This Document

This document is the single source of truth for building the FamilyWallet mobile frontend. Every screen, every navigation transition, every popup, every API call, and every error state is described in full so that an AI assistant (or developer) can build the React Native app without ambiguity.

It is derived from two sources:

- FamilyWallet PRD v1.0 — product requirements and business rules
- FamilyWallet API Postman Collection — the actual backend endpoints, request bodies, and response shapes

Read this document top-to-bottom before writing any screen. Every screen references the API endpoints it must call, the navigation decisions that follow each response, and the exact modals or toasts to show.

1. Global Rules for All Screens

1.1 Authentication Token

After a successful login, the API returns an `access_token` and a `refresh_token` inside `data.token`. Store both securely (e.g. in `SecureStore` on Expo or `Keychain`). Attach the `access_token` to every subsequent request as:

```
Authorization: Bearer <access_token>
```

If any protected endpoint returns HTTP 401, clear the stored tokens and navigate the user to the Login screen.

1.2 API Base URL

All endpoints use the variable `{{baseUrl}}`. In development this is `http://localhost:3000`. In production, replace with the deployed Railway/Render URL. Never hard-code it — store it in a config file.

1.3 Toast vs Modal

Pattern	When to use it
Toast (bottom, auto-dismiss 3s)	Non-critical feedback: purchase confirmed, member invited, limit updated
Inline field error	Validation failures on input fields (empty, wrong format)
Bottom Sheet / Modal	Confirmation before destructive actions (freeze member, buy now, fund wallet)

Full-screen error state	Network failure or server 500 — show retry button
-------------------------	---

1.4 Loading States

Every button that triggers an API call must show a loading spinner inside it and disable itself while the call is in-flight. Never let the user double-tap and double-submit.

1.5 accountType Drive Navigation

The field `accountType` in the login response determines which dashboard the user lands on after login. The three valid values are:

- `solo` → Solo Dashboard
- `group_head` → Group Head Dashboard
- `member` → Member Dashboard

Cache the `accountType` in app state. Use it to conditionally show/hide Group Head controls throughout the app.

2. App Screen Map

Below is every screen in the MVP and which user types can access it.

Screen	Solo	Group Head	Member
Splash / Welcome	✓	✓	✓
Registration	✓	✓	✓
OTP Verification	✓	✓	✓
Login	✓	✓	✓
Forgot Password	✓	✓	✓
Reset Password (OTP + new password)	✓	✓	✓
Solo Dashboard	✓	—	—
Group Head Dashboard	—	✓	—
Member Dashboard	—	—	✓
Buy Data Screen	✓	✓	✓
Buy Airtime Screen	✓	✓	✓
Fund Wallet Screen	✓	✓	—
Create Group Screen	—	✓ (first time)	—
Member Management Screen	—	✓	—
Add Member Screen	—	✓	—
Transaction History Screen	✓	✓	✓
Profile & Settings Screen	✓	✓	✓

3. Auth Screens

3.1 Splash / Welcome Screen

This is the first screen the user sees when they open the app.

What is on this screen

- FamilyWallet logo and tagline: "One wallet. Every bill. Every person in your life."
- Two CTA buttons: Create Account and Log In
- No API calls on this screen

Navigation decisions

- Tap Create Account → navigate to Registration Screen
- Tap Log In → navigate to Login Screen
- On app launch: check if a valid access_token exists in SecureStore. If yes, silently call GET /api/v1/auth/profile. If successful, navigate directly to the correct Dashboard based on accountType. If 401, clear token and show this screen normally.

3.2 Registration Screen

What is on this screen

- Fields: First Name, Last Name, Email, Phone Number (Nigerian format, e.g. 08012345678), Password, Confirm Password
- Submit button: Create Account
- Link at bottom: Already have an account? Log In

API call on submit

```
POST /api/v1/auth/register  
{ firstName, lastName, email, phone, password, confirmPassword }
```

Outcomes

- Success (200): Navigate to OTP Verification Screen. Pass the email forward as a route parameter so OTP screen knows where to submit.
- Validation error (400): Show inline field errors. Do not navigate.
- Duplicate email/phone (409 or similar): Show inline error under the relevant field.

 *Client-side validate before calling the API: all fields required, passwords must match, phone must be 11 digits starting with 0.*

3.3 OTP Verification Screen

What is on this screen

- Heading: Verify Your Account
- Subtext: "We sent a 6-digit code to your phone and email."
- Six individual digit input boxes (or one 6-digit OTP input field)
- Button: Verify
- Link: Resend OTP

API call on Verify

```
POST /api/v1/auth/activate
{ email: <from route params>, otp: <entered digits> }
```

Outcomes

- Success (200): Show toast "Account activated! Please log in." → Navigate to Login Screen.
- Wrong OTP (400/401): Show inline error under OTP input: "Invalid or expired code. Try again."
- Resend OTP tapped: Call **POST** /api/v1/auth/forgot-password with the user's email (this triggers a new OTP via the same Sendchamp route). Show toast "A new code has been sent."

 *Auto-submit the OTP when all 6 digits are filled. Do not require the user to tap Verify manually.*

3.4 Login Screen

What is on this screen

- Fields: Email, Password
- Link: Forgot Password?
- Button: Log In
- Link at bottom: Don't have an account? Sign Up

API call on Log In

```
POST /api/v1/auth/login
{ email, password }
```

Success response — key fields to store

- data._id — user ID
- data.accountType — "solo" | "group_head" | "member"
- data.walletId
- data.token.access_token — store in SecureStore
- data.token.refresh_token — store in SecureStore
- data.firstName, data.lastName — for display in Dashboard header

Navigation after success

- accountType === "solo" → Solo Dashboard
- accountType === "group_head" → Group Head Dashboard
- accountType === "member" → Member Dashboard

Failure outcomes

- Wrong credentials (401): Show inline error under password field: "Email or password is incorrect."
- Account not activated (403 or similar): Show modal prompting user to verify their account, with a button that navigates to OTP Verification Screen.

3.5 Forgot Password Screen

What is on this screen

- Field: Email
- Button: Send Reset Code

API call

```
POST /api/v1/auth/forgot-password
{ email }
```

Outcomes

- Success: Show toast "A reset code has been sent to your email." → Navigate to Reset Password Screen, passing email as a route param.
- Email not found: Show inline error under email field.

3.6 Reset Password Screen

What is on this screen

- 6-digit OTP input
- Fields: New Password, Confirm New Password
- Button: Reset Password

API call

```
POST /api/v1/auth/reset-password
{ email: <from route params>, otp, newPassword, confirmPassword }
```

Outcomes

- Success: Show toast "Password changed successfully." → Navigate to Login Screen.
- Wrong OTP: Inline error under OTP field.
- Passwords don't match: Inline error under confirmPassword field.

4. Solo Dashboard

This is the home screen for accountType === "solo". It is the first screen after login for solo users.

4.1 What is on this screen

- Header: "Hello, {firstName}" with a notification bell icon (top right, inactive in MVP)
- Wallet balance card — shows current balance in NGN
- Quick action buttons row: Buy Data | Buy Airtime | Fund Wallet
- Recent Transactions — last 5 transactions, each showing service type, network, amount, date
- Bottom navigation bar: Home | Transactions | Settings

4.2 API calls on screen load

Call both of these on mount:

```
GET /api/v1/wallet/balance
GET /api/v1/auth/profile
```

Use the balance from wallet/balance for the wallet card. Use firstName/lastName from profile for the header greeting.

 Show a skeleton loader over the balance card while the API call is in-flight. Do not show ₦0 before data loads.

4.3 Navigation decisions

Screen / State	Trigger	Behaviour
----------------	---------	-----------

Tap Buy Data	User action	Navigate to Buy Data Screen. Pass context: { userType: 'solo' }
Tap Buy Airtime	User action	Navigate to Buy Airtime Screen. Pass context: { userType: 'solo' }
Tap Fund Wallet	User action	Navigate to Fund Wallet Screen
Tap a recent transaction	User action	Navigate to Transaction History Screen, scrolled to that transaction
Tap Transactions in bottom nav	User action	Navigate to Transaction History Screen
Tap Settings in bottom nav	User action	Navigate to Profile & Settings Screen
Balance falls below ₦2,000	API response from balance check	Show a persistent banner (not a toast) at the top of the screen: "Low wallet balance. Top up to continue buying." with a Fund Wallet button

5. Group Head Dashboard

This is the home screen for accountType === "group_head".

5.1 What is on this screen

- Header: "Hello, {firstName}" with notification bell
- Group wallet balance card — shows the group wallet balance
- Member list — each card shows: member name, spend limit, amount spent this month, status (active/frozen)
- Quick action buttons: Add Member | Fund Wallet | Buy for Member
- Bottom navigation bar: Home | Members | Transactions | Settings

5.2 First-time flow (no group exists yet)

After login, immediately call GET /api/v1/auth/profile. If the profile has no groupId or the groups array is empty, intercept navigation and show the Create Group Screen as a mandatory step before showing the dashboard. The user must create a group before they can do anything else as a group head.

5.3 API calls on screen load

GET /api/v1/wallet/balance?groupId=<groupId>

GET /api/v1/auth/profile

The groupId comes from the profile response. Store it in app state after login.

5.4 Navigation decisions

Screen / State	Trigger	Behaviour
Tap Add Member	User action	Navigate to Add Member Screen
Tap Fund Wallet	User action	Navigate to Fund Wallet Screen with groupId in context
Tap a member card	User action	Open Member Detail Bottom Sheet (see 5.5)
Tap Members in bottom nav	User action	Navigate to Member Management Screen

Tap Transactions in bottom nav	User action	Navigate to Transaction History Screen
Group balance < ₦2,000	API response	Show persistent top banner with Fund Wallet shortcut
No group created yet	Profile check on mount	Intercept → Create Group Screen (mandatory)

5.5 Member Card — Bottom Sheet

When a group head taps a member card, a bottom sheet slides up. It contains:

- Member name and phone
- Spend limit field (editable — shows current value)
- Button: Update Limit
- Button: Freeze Member (if active) / Unfreeze Member (if frozen) — show in red
- Button: Buy Data for this member
- Button: Buy Airtime for this member

Update Limit taps calls:

```
PATCH /api/v1/group/member/limit
{ groupId, newLimit }
```

- Success: Show toast "Spending limit updated." Refresh member list.

⚠ *Freeze/Unfreeze is a v1.1 feature. Render the button but show a "Coming soon" toast if tapped in MVP.*

6. Member Dashboard

This is the home screen for `accountType === "member"`. Members spend from the group wallet within their assigned limit.

6.1 What is on this screen

- Header: "Hello, {firstName}"
- Sub-balance card: shows the member's remaining limit for the month (`spend_limit` minus `spent_this_month`)
- Quick actions: Buy Data | Buy Airtime
- Recent personal transactions (last 5)
- Bottom navigation: Home | Transactions | Settings

6.2 API calls on screen load

```
GET /api/v1/wallet/balance?groupId=<groupId>
GET /api/v1/auth/profile
```

6.3 Navigation decisions

Screen / State	Trigger	Behaviour
Tap Buy Data	User action	Navigate to Buy Data Screen with context: { userType: 'member', groupId }
Tap Buy Airtime	User action	Navigate to Buy Airtime Screen with context: { userType: 'member', groupId }

Sub-balance = ₦0 or frozen	API data	Disable Buy Data and Buy Airtime buttons. Show label: "Limit reached. Contact your group head."
Tap Transactions in bottom nav	User action	Transaction History Screen (member sees only their own transactions)
Tap Settings in bottom nav	User action	Profile & Settings Screen

7. Buy Data Screen

This screen is shared between Solo, Group Head, and Member users. Context passed via route params determines how the purchase is attributed.

7.1 What is on this screen

- Step 1 — Network selector: 4 network tiles (MTN, Airtel, GLO, 9mobile). These are loaded from the API.
- Step 2 — Plan selector: appears after network is chosen. Shows a list of available data plans as selectable cards.
- Step 3 — Phone number input: text field pre-filled with the user's own phone, with option to change.
- Purchase summary bar at bottom: shows selected plan + price.
- Button: Buy Now

7.2 API sequence

Step 1 — On screen load, fetch networks for data:

```
GET /api/v1/utility/networks?type=data
```

Render the returned names as network selector tiles. Store the identifier field for use in Step 2.

Step 2 — When user taps a network tile:

```
GET /api/v1/utility/data-plans?networkId=<identifier>
```

Render each plan as a card: label (e.g. "2GB = ₦1,100 (30 Days)") and amount. Show a loading spinner on the plan list area while fetching.

Step 3 — When user taps Buy Now, show a Confirmation Bottom Sheet (see 7.3).

7.3 Confirmation Bottom Sheet

Before purchase, slide up a sheet showing:

- Plan: 2GB — 30 Days
- Network: MTN (Data Share)
- Recipient: 08012345678
- Amount: ₦1,100
- Two buttons: Cancel (dismisses sheet) | Confirm Purchase

7.4 API call on Confirm

```
POST /api/v1/purchase/buy
```

```
{ service: "data", amount, recipient, network, bundleId: plan_code, groupId: <if member or group head purchase> }
```

The groupId field is optional. Include it when:

- The logged-in user is a member (always pass their groupId)

- The group head is buying for a member (pass groupId)
- The solo user is buying for themselves (omit groupId entirely)

7.5 Outcomes

- Success: Dismiss sheet. Show full-screen success state: green checkmark, plan name, amount deducted, recipient number. Add a button: Back to Home. Do not auto-navigate — let the user control.
- Insufficient balance: Show modal: "Insufficient wallet balance." with a Fund Wallet button (navigates to Fund Wallet Screen) and a Cancel button.
- Member limit exceeded: Show modal: "You have reached your monthly limit. Contact your group head."
- Network/API error: Show full-screen error with retry button.

 *The network identifier values from the networks endpoint (e.g. mtn_data_share, mtn_gifting_data) are the networkId for fetching plans, but the network field in the purchase payload should be the plain network name (MTN, AIRTEL, GLO, 9mobile). Confirm this mapping with the backend developer before going live.*

8. Buy Airtime Screen

8.1 What is on this screen

- Network selector tiles: MTN, Airtel, GLO, 9mobile
- Amount input field — numeric keyboard, minimum ₦50
- Phone number input — pre-filled with user's phone, editable
- Buy Now button

8.2 API call — networks

GET /api/v1/utility/networks?type=airtime

This returns the 4 networks as { id, name }. Render as tiles. The id (e.g. "mtn") maps to the network field in the purchase payload.

8.3 Confirmation Bottom Sheet

- Network: MTN
- Amount: ₦500
- Recipient: 08012345678
- Cancel | Confirm Purchase

8.4 API call on Confirm

POST /api/v1/purchase/buy

{ service: "airtime", amount, recipient, network, groupId: <optional> }

Note: no bundleId for airtime purchases.

8.5 Outcomes

Identical to Buy Data outcomes: success state, insufficient balance modal, limit exceeded modal, error state.

9. Fund Wallet Screen

Available to Solo users and Group Heads only. Members cannot fund the wallet.

9.1 What is on this screen

- Amount input — numeric keyboard, minimum ₦500
- Fund Wallet button
- Note: "A 1.5% + ₦100 Paystack processing fee applies."

9.2 API call on Fund Wallet tap

```
POST /api/v1/wallet/fund/init
{ amount }
```

The backend calls Paystack's transaction/initialize endpoint and returns a Paystack authorization URL or inline payment data.

9.3 After initialization

- Open the Paystack payment flow. On React Native, use the @paystack/react-native package or open the authorization URL inside a WebView.
- Listen for the Paystack callback. On success, the Paystack webhook fires to POST /api/v1/webhook/paystack on the backend — this is handled server-side. The app should not try to manually confirm the payment.
- After the WebView closes (or Paystack callback fires), navigate back to the Dashboard and refresh the wallet balance. If the balance has updated, show a toast: "Wallet funded successfully."
- If the user closes the WebView without completing payment: show toast "Funding cancelled." and stay on Fund Wallet Screen.

 *The groupId field in the webhook metadata is set by the backend when the group head funds the group wallet. The frontend does not need to send groupId to /wallet/fund/init — the backend determines this from the JWT token and whether the user is a group head.*

10. Create Group Screen

This screen appears for a Group Head who has not yet created a group. It is a mandatory step — they cannot access the Group Head Dashboard until it is complete.

10.1 What is on this screen

- Field: Group Name (e.g. "The Doe Family")
- Dropdown / selector: Group Type — Family | Household | Guardian | Small Business | Community | School Parent
- Button: Create Group

10.2 API call

```
POST /api/v1/group/create
{ groupName, groupType }
```

10.3 Outcomes

- Success: Store the returned groupId in app state and AsyncStorage. Navigate to Group Head Dashboard.

- Error: Show inline error. Do not navigate.
-

11. Member Management Screen

Accessible from the Group Head Dashboard's bottom nav (Members tab).

11.1 What is on this screen

- List of all group members: name, phone, spend limit, amount spent this month, status badge (Active / Frozen)
- Floating action button: + Add Member
- Tap any member row → opens Member Detail Bottom Sheet (same as described in section 5.5)

11.2 No API endpoint in collection for listing members

The current Postman collection does not include a GET /group/members endpoint. However, the profile endpoint (GET /api/v1/auth/profile) may return member data for a group head. Confirm with the backend developer. If a separate endpoint exists, call it on mount and on return from Add Member Screen.

12. Add Member Screen

12.1 What is on this screen

- Field: Member Email
- Field: Member Phone Number
- Field: Monthly Spend Limit (฿ amount)
- Button: Add Member

12.2 API call

```
POST /api/v1/group/member/add
{ groupId: <from app state>, email, phone, spendLimit }
```

12.3 Outcomes

- Success (200): Show toast "Member invited. They will receive an SMS with the app link." → Navigate back to Member Management Screen and refresh the member list.
 - Member not found / already exists (400/409): Show inline error under email field.
 - Missing groupId: This is an app error — should never happen. Log and show generic error.
-

13. Transaction History Screen

Available to all user types. What the user sees is filtered by their role.

13.1 What is on this screen

- Filter bar: All | Airtime | Data — toggles to filter by service type
- Transaction list: each item shows service icon, network, amount, recipient number, date/time, status badge

- For Group Heads: additional filter by member name

13.2 API note

The Postman collection does not include a transaction history endpoint in the current build. This endpoint should follow the pattern GET /api/v1/purchase/history or GET /api/v1/transactions. Confirm the endpoint with the backend developer. Apply the groupId query param for group-scoped views.

14. Profile & Settings Screen

14.1 What is on this screen

- User avatar (initials-based, no image upload in MVP)
- Name and email (read only, from profile API)
- Phone number (read only)
- Account type badge: Solo / Group Head / Member
- Settings rows: Change Password (navigates to Forgot Password flow) | Notifications (toggle, stored locally) | Log Out

14.2 Log Out behaviour

- Show confirmation modal: "Are you sure you want to log out?"
 - On confirm: clear access_token and refresh_token from SecureStore. Clear app state. Navigate to Splash / Welcome Screen.
-

15. Error States & Edge Cases

15.1 No Internet Connection

Detect network status using NetInfo. When offline, show a persistent banner: "No internet connection." Disable all API-triggering buttons. Retry automatically when connection is restored.

15.2 Session Expiry

If a 401 is returned from any protected endpoint, treat it as session expiry. Clear all stored tokens, show a modal: "Your session has expired. Please log in again." → Navigate to Login Screen.

15.3 Peyflex API Downtime

If the buy endpoint returns a Peyflex-related error, show: "This service is temporarily unavailable. Please try again shortly." Do not show a generic "Something went wrong" — users trust the app more when error messages are specific.

15.4 Empty States

- Transaction History — no transactions yet: show illustration and text "No transactions yet. Start by buying data or airtime."
 - Member list — no members yet: show illustration and text "Your group is empty. Add your first member." with an Add Member button.
-

16. Quick API Reference

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Create account
POST	/api/v1/auth/activate	Verify OTP, activate account & create wallet
POST	/api/v1/auth/login	Login, receive JWT tokens
POST	/api/v1/auth/forgot-password	Request password reset OTP
POST	/api/v1/auth/reset-password	Reset password with OTP
GET	/api/v1/auth/profile	Get current user profile
POST	/api/v1/group/create	Create a group (Group Head only)
POST	/api/v1/group/member/add	Add a member to the group
PATCH	/api/v1/group/member/limit	Update a member's spend limit
POST	/api/v1/wallet/fund/init	Initialize Paystack wallet funding
GET	/api/v1/wallet/balance	Get wallet balance (pass groupId for group wallet)
POST	/api/v1/purchase/buy	Buy airtime or data
GET	/api/v1/utility/networks?type=airtime data	Get available networks
GET	/api/v1/utility/data-plans?networkId=...	Get data plans for a network
POST	/api/v1/webhook/paystack	Paystack payment webhook (server-side only, not called by app)